

Exemplo - Módulo 2: Arquiteturas Single-Agent (resolvido com TrialForge)

Disciplina: Arquitetura de Sistemas com IA · Prof. Ahirton Lopes, Ph.D.

Missão Prática #02 - solução de referência

Caso escolhido: geração da seção de assentimento (assentimento de menores) de um ICF, quando o protocolo do estudo envolve participantes de 12 a 17 anos.

Passo 1 - Protótipo calibrado

- Memória: MÍNIMA, sem estado entre protocolos. Justificativa: cada estudo clínico é avaliado de forma isolada - cláusulas regulatórias mudam de versão, e o agente 'lembrar' de um estudo anterior corre o risco de aplicar uma cláusula desatualizada por engano.
- Loop (ReAct): SIM, máximo 4 voltas. Justificativa: a tarefa exige decisão condicional (o estudo envolve menores?) antes de buscar o conteúdo certo - na prática, resolve em 2 voltas: uma para checar a condição, outra para buscar a cláusula.
- Reflexão: Não incluída neste protótipo de referência. Justificativa: decidir se a seção se aplica (é sobre menores?) e buscar a cláusula certa é direto o bastante para não exigir uma segunda passada crítica - qualquer erro que escapar é pego na revisão do Approval Gate, antes de qualquer aprovação.
- Ferramentas: uma - buscar_clausula_regulatoria. É a única porta de saída do agente para o mundo real nesta tarefa.

Passo 2 - Schema da ferramenta

Nome: buscar_clausula_regulatoria

Descrição: "Busca cláusulas regulatórias de estudos clínicos por tema e jurisdição. Use quando precisar de texto normativo (ANVISA ou FDA) para compor uma seção do documento."

Parâmetros:

- tema (string) - ex: "assentimento_menor", "teste_hiv"
- jurisdicao (enum: "ANVISA" | "FDA") - restrito às duas opções válidas no escopo do projeto

Retorno: { texto: string, fonte: string (ex: "RDC ANVISA 466/2012, Art. 4º") }

Classificação: ferramenta de LEITURA - não altera nenhum estado real, portanto não precisa passar pelo Approval Gate antes de executar. Só a seção final gerada, depois de reflexão, passa pelo Gate.

Passo 3 - Critério de parada

Limite: máximo 4 iterações do loop.

O que acontece no limite: se o agente não convergir em 4 voltas - por exemplo, se a busca de cláusula devolver resultado vazio repetidamente - o loop é interrompido automaticamente pelo

Orquestrador, e a tarefa é encaminhada ao Approval Gate com a última Observação registrada, sinalizando que a automação não foi suficiente. Isso nunca falha silenciosamente.

Passo 4 - Implementação em JavaScript

Ver `react-agent-prototype.js`, na mesma pasta de demos - implementa exatamente este protótipo calibrado: loop ReAct de até 4 voltas, uma ferramenta (`buscar_clausula_regulatoria`), sem reflexão de segundo nível nesta versão de referência (decisão consciente de manter o protótipo mínimo), sempre encaminhando o rascunho final ao Approval Gate antes de qualquer aprovação. Existe versão equivalente em Python (`react_agent_prototype.py`) como material de apoio - a entrega oficial da Missão Prática é em JavaScript, conforme a ementa.

Nota de curadoria

Repare que os quatro componentes deste protótipo somam pouca complexidade - de propósito. Uma seção condicional de ICF não precisa de memória de longo prazo, nem de múltiplas ferramentas, nem de reflexão de conteúdo em toda resposta. A calibragem correta é sempre a mínima que a tarefa exige, não a máxima disponível.

Ahirton Lopes · AI Architecture Toolkit - UNIPDS: Arquitetura de Sistemas com IA

Prof. Ahirton Lopes, Ph.D. - GDE AI, Microsoft MVP, Senior Manager